

Efficient Data Storage Systems

University of Minho

Guide 0

Warm-up

2026

Warm-up

This guide aims to set up the environment for the Data Storage Systems laboratory. It also serves as a brief revision of data structures, threading, and concurrency control in the C programming language.

1 Setup

1.1 VirtualBox Installation

Start by installing the VirtualBox tool on your machine and setting up the network, with the following steps:

1. Download the appropriate version of **VirtualBox 7.2.4** for your OS: https://www.virtualbox.org/wiki/Download_Old_Builds_7_2
2. Create a new Host-only network adapter with the following steps:
 - (a) Navigate to the Network Manager page (e.g., **File > Tools > Network Manager**)
 - (b) On the “*Host-only Networks*” tab, create a new network (by clicking the “**Create**” button)
 - (c) In Windows/Linux, ensure the Network adapter's IP is set to “192.168.60.1”, the DHCP Server's IP is set to “192.168.60.2”, “*Lower Bound*” is set to “192.168.60.201” and the “*Upper Bound*” is set to “192.168.60.254”. In MacOS adjust the bounds to the same values.

1.2 Vagrant installation

Now, install the Vagrant tool with the following steps:

1. Download and install Vagrant (**version 2.4.9**) at: <https://developer.hashicorp.com/vagrant/install>
2. After verifying the Vagrant installation with `vagrant --version`

1.3 Vagrantfile

Vagrant uses a configuration file (*Vagrantfile*) to describe the type of machines required for a project and how to configure and provision them. Inspect and adapt the Vagrantfile provided along with this guide.

1. The file describes how to configure and provision VMs with 2 GiB of RAM and 2 virtual CPUs.
2. In the beginning of the file, several variables are defined:

- (a) The local path for the user's SSH public and private keys, and the content of the public key. SSH keys are used to authenticate a user or process that wants to access a remote system through the SSH protocol. We need to share our public key with the VMs so we can access them from our computer.

```
PUBLIC_KEY_PATH = "~/ssh/id_rsa.pub"
PRIVATE_KEY_PATH = "~/ssh/id_rsa"
READ_PUBLIC_KEY = File.read(File.expand_path(PUBLIC_KEY_PATH)).strip
```

- (b) The number of VMs to create and the IP range of each VM's private network

```
Number_Nodes = 1
NODES_IP_RANGE= "192.168.60"
```

Note 1: SSH key variables may need to be adapted for your deployment.

Note 2: Vagrant allows calling ruby code. The `READ_PUBLIC_KEY` variable stores the content of the public key found at `PUBLIC_KEY_PATH`.

3. The Linux distribution (*i.e.*, VM base image), the base configuration (*i.e.*, 2 GiB RAM and 2 vCPUs) and the provision tasks that should execute after starting the VM are defined as global configurations:

```
config.vm.box = "bento/ubuntu-24.04"
config.vm.provider "virtualbox" do |vb|
  vb.memory = 2048
  vb.cpus = 2
end
config.vm.provision :shell, privileged: true, inline: $provision_all
```

4. The `provision_all` script used in the `config.vm.provision` parameter specifies a *shell* environment to execute the commands necessary for configuring the SSH Public Key authentication and to install required Ubuntu packages.

```
$provision_all = <<-SHELL
echo "[ALL|Task 1] Configure SSH Public Key authentication"
echo "#{READ_PUBLIC_KEY}" >> /home/vagrant/.ssh/authorized_keys
(...)
apt update -y
apt install build-essential -y
apt install libgl2.0-dev -y
SHELL
```

Note 1: The second command copies the content of the public SSH key (stored at the `READ_PUBLIC_KEY` variable) to the `authorized_keys` file at the VM's file system.

Note 2: All the previous commands execute inside the VM!

5. Specific configurations for each VM are then specified independently. The example below uses a loop to change the hostname and the IP address for each *node* VM:

```
(1..Number_Nodes).each do |i|
  config.vm.define "node#{i}" do |node|
    node.vm.hostname = "node#{i}"
    node.vm.network :private_network, ip: "#{NODES_IP_RANGE}.#{100+i}"
  end
end
```

Note 1: A private network is only accessible for the host and other VMs running at the host. A NAT public network is used for communication between VMs and the internet.

Note 2: Inspect the configuration for *node* VMs. What IP addresses are being created for these VMs?

1.4 Testing

1. Deploy the VM by running `vagrant up` (in the same folder where the Vagrantfile is stored).
2. Check if the VM was created with `vagrant status`.
3. Connect to the VM by using SSH (the default username is `vagrant`). You should use the standard SSH command in your shell. **Do not use the `vagrant ssh` command!**
4. Check (with the `ip add` command) that the VMs have a public (NAT network created automatically by Vagrant) and private network interface configured.

Note 1: To power off the VM use `vagrant halt`. To destroy the VM use `vagrant destroy`.

2 Exercises

The goal is to implement an index that stores metadata (e.g., size, reference count) of files, which are identified by unique filenames. Use the provided base code to explore the index's API (`metaindex.h`) and potential workloads for testing this component (`metanode.c`)

2.1 Implementing the index

Explore the Glib Hashtable API at <https://docs.gtk.org/glib/data-structures.html> to implement the index functionality (file `metaindex.c`).

Warning: Note that neither keys nor values are copied when inserted into the `GHashTable`, so they must exist for the lifetime of the `GHashTable`. In other words, memory management must be done by the program.

Hint: Take a look at the following functions to implement the desired functionality:

- `g_hash_table_new`
- `g_hash_table_new_full`
- `g_hash_table_contains`
- `g_hash_table_insert`
- `g_hash_table_lookup`
- `g_hash_table_remove`
- `g_hash_table_destroy`

2.2 Threads, Mutexes and Conditions

To ensure thread safety, explore the provided code and refer to <https://www.cs.cmu.edu/afs/cs/academic/class/15492-f07/www/pthreads.html> for a short tutorial on multi-threading in C (namely, threads, mutexes, and condition variables).

1. Inspect the `metanode.c` file to understand how threads are created and terminated. Run the program with 10 threads (`#define N_THREADS`). Is the output of the program correct?
2. Use the mutex in the `Index` struct to ensure thread safety.
3. Implement the `index_wait_refs` (`metanode.c`) function, which should block the execution until a file's reference count reaches a specified threshold. Use the condition variable provided in the `Index` struct.

Validation: Create a new workload at the `metanode.c` file that spawns an independent thread to execute this function.

3 Extras

1. The current implementation uses a single mutex for all operations. Is there a way to optimize this?
2. All threads invoking `index_wait_refs` are awoken even when these are waiting for distinct keys. Is there a way to optimize this?
3. Implement `index_multi_inc_refs`, which increments the reference count (by 1) of a set of keys received as argument. The operation should be thread-safe (*i.e.*, other threads are only able to see the new reference count values when all the keys have been updated).
4. Explore other Glib data structures such as queues, linked lists, and trees.

Learning outcomes

Experiment Linux virtual machines. Explore the Glib API and HashTable structure. Understand threading, mutex, and condition variables in the C programming language.